HEALTHCARE + FITNESS AGENT

Production-Grade Developer Guide

End-to-End Implementation · Docker Deployment · Azure-Ready

Field	Value
Version	1.0.0 — DEV MODE
Stack	FastAPI · LangGraph · FAISS · SQLite · React · Docker
Phase	Phase 1 — Local Docker Container
Next Phase	Phase 2 — Azure Container Instances / ACI
Classification	Internal — Engineering
Date	April 25, 2026

1. Executive Summary

This guide is the authoritative implementation reference for the Healthcare + Fitness Agent — a multi-agent AI system that ingests medical lab reports (PDFs), analyzes health parameters, generates risk insights, and produces personalized fitness and diet plans. The system runs fully offline in Phase 1 (local Docker), with all AI inference delegated to Azure OpenAI.

The platform is built on a modern, modular Python/TypeScript stack. Every component is independently testable, containerized, and designed for future horizontal scaling. Engineers can have a fully functional local environment running within 30 minutes of cloning the repository.

> **Design Philosophy**
> Stateless-but-structured: each API request carries all needed context. Per-user data isolation via user_id namespacing. All AI calls are prompt-templated and guarded with medical safety disclaimers. No external dependencies except Azure OpenAI.

2. System Architecture

2.1 High-Level Overview

The system follows a clean 6-tier architecture enclosed within a single Docker Compose stack. Requests flow strictly top-down; responses flow bottom-up. The only external network dependency is the Azure OpenAI API.

Tier 1 — Browser / Client: React SPA served by Nginx on port 3000. Communicates with FastAPI via REST and WebSocket.

Tier 2 — FastAPI Backend: Python 3.11 async API on port 8000. Handles routing, file uploads, authentication scaffolding, and orchestrator delegation.

Tier 3 — LangGraph / LangChain Orchestrator: Stateful agent graph that routes user intent to the correct specialized agent, maintains conversation memory, and coordinates RAG retrieval.

Tier 4 — Agent Layer: Five specialized LangGraph agents, each with a defined input schema, prompt template, and output contract. Agents are independently callable and composable.

Tier 5 — Storage Layer: FAISS vector index (per-user namespaced), SQLite relational database, and local PDF filesystem store. Zero cloud storage in Phase 1.

External — Azure OpenAI: GPT-4o for all text generation and reasoning. text-embedding-3-small for embedding generation. All calls are rate-limited and retried.

2.2 Internal Component Map

```
healthcare-agent/
  backend/                    FastAPI application root
    app.py                    Entry point — Uvicorn server
    routes/
```

```
    upload.py                PDF upload endpoint
    chat.py                  RAG chat endpoint
    insights.py              Health insight retrieval
    fitness.py               Fitness plan generation
  services/
    pdf_service.py           Ingestion orchestration
    embedding_service.py     FAISS index management
    risk_service.py          Risk scoring engine
    fitness_service.py       Plan generator
  agents/
    intake_agent.py          BMI / vitals intake
    doc_agent.py             PDF parameter extraction
    risk_agent.py            Health risk classification
    disease_agent.py         Disease pattern detection
    fitness_agent.py         Plan generation agent
    graph.py                 LangGraph StateGraph wiring
  utils/
    pdf_parser.py            PyMuPDF text extraction
    prompt_templates.py      All LLM prompt templates
    guardrails.py            Medical safety wrappers
  db/
    database.py              SQLite connection & session
    models.py                SQLAlchemy ORM models
    init_db.py               Schema creation script
  models/
    schemas.py               Pydantic v2 request/response
frontend/                    React + Vite application
  src/
    components/              Reusable UI components
    pages/                   Route-level page components
    services/api.ts          Typed API client
    store/                   Zustand state management
data/
  pdfs/                      Uploaded PDF storage
  faiss_index/               Per-user FAISS indexes
  drug_knowledge/            Drug interaction text files
config/
  settings.py                Pydantic BaseSettings config
  .env                       Environment variables (git-ignored)
docker/
  Dockerfile.backend         Python backend image
  Dockerfile.frontend        Nginx + React image
  nginx.conf                 Nginx proxy configuration
docker-compose.yml           Full stack orchestration
requirements.txt             Python dependencies
requirements.dev.txt         Dev/test dependencies
```

3. Complete Tech Stack

Layer	Technology	Purpose / Notes
Frontend	React 18.3	UI framework — Vite build, TypeScript strict mode
Frontend	Tailwind CSS 3.x	Utility-first styling with design tokens
Frontend	Zustand	Lightweight client state management

Frontend	Axios + React Query	HTTP client with caching and invalidation
API Layer	FastAPI 0.111	Async Python REST API with OpenAPI auto-docs
API Layer	Uvicorn	ASGI server — single-worker in dev, multi-worker in prod
API Layer	Pydantic v2	Request/response validation and serialization
Orchestration	LangChain 0.2+	LLM chains, prompts, memory, retrievers
Orchestration	LangGraph	Stateful multi-agent graph with typed state
LLM	Azure OpenAI	GPT-4o (chat) + text-embedding-3-small (embeddings)
Vector DB	FAISS	Local vector similarity search — CPU-only dev mode
Relational DB	SQLite + SQLAlchemy	Structured health records — zero-config
PDF Parsing	PyMuPDF (fitz)	Fast text extraction from lab report PDFs
PDF Parsing	pdfplumber	Table-aware extraction for structured lab data
File Storage	Local filesystem	PDF archive in data/pdfs/ — mountable volume
Security	python-jose + passlib	JWT auth scaffold for future production use
Testing	pytest + httpx	Unit + integration tests with async support
Containerization	Docker + Compose	Phase 1: local containers; Phase 2: Azure ACI
Proxy	Nginx	Static serving + reverse proxy to FastAPI
Config	python-dotenv	12-factor app environment variable management
Logging	structlog	Structured JSON logging for observability

4. Data Flow — Step by Step

4.1 PDF Ingestion Flow

1. User uploads a PDF via the React UI (POST /api/upload)
2. FastAPI receives the multipart/form-data, validates file type and size
3. pdf_service saves the file to data/pdfs/{user_id}_{timestamp}.pdf
4. PyMuPDF (fitz) opens the file and extracts raw text page-by-page
5. pdfplumber is invoked for any page containing tabular data (lab grids)
6. The combined raw text is passed to the Document Analysis Agent
7. Doc Agent sends a structured extraction prompt to Azure OpenAI GPT-4o
8. GPT-4o returns a JSON array of {test_name, value, unit, reference_range}
9. Each extracted parameter is stored as a row in SQLite health_records
10. The full extracted text is chunked into 500-token overlapping segments

11. Each chunk is embedded via Azure OpenAI text-embedding-3-small
12. Embeddings are upserted into the user's FAISS index (data/faiss_index/{user_id}/)
13. FastAPI returns a success response with a summary of extracted parameters

4.2 Chat / RAG Flow

14. User sends a health question via the chat UI (POST /api/chat)
15. FastAPI receives {user_id, session_id, query} and delegates to the RAG chain
16. LangGraph loads the user's FAISS index and creates a retriever (top-k=5)
17. The user query is embedded and similarity-searched against FAISS
18. Top-5 most relevant text chunks are retrieved as context
19. LangGraph builds a RetrievalQA prompt: [system] + [context] + [user query]
20. The prompt is sent to Azure OpenAI GPT-4o
21. GPT-4o generates a response grounded in the retrieved health data
22. The guardrails module appends the medical disclaimer to every response
23. The response is streamed back to the React UI via Server-Sent Events

4.3 Insights & Plan Generation Flow

24. User requests insights (GET /api/insights/{user_id})
25. FastAPI queries SQLite for all health_records belonging to user_id
26. Risk Scoring Engine evaluates each parameter against reference ranges
27. Disease Pattern Agent checks multi-parameter combinations (e.g. metabolic syndrome)
28. Scores are aggregated into a composite risk_level: Low / Moderate / High / Critical
29. User requests fitness plan (POST /api/fitness-plan)
30. Fitness Agent receives {age, bmi, risk_level, restrictions} as structured input
31. Agent builds a structured prompt and calls Azure OpenAI GPT-4o
32. GPT-4o returns {workout, diet, restrictions, lifestyle} JSON
33. FastAPI validates the response with Pydantic and returns it to the client

5. Prerequisites & Environment Setup

5.1 System Requirements

- Docker Desktop 4.28+ with Docker Compose v2 (docker compose — note: no hyphen)
- macOS 13+ / Ubuntu 22.04+ / Windows 11 with WSL2
- 4 GB RAM minimum allocated to Docker (8 GB recommended)
- Azure OpenAI resource with GPT-4o and text-embedding-3-small deployed

- Git 2.40+

5.2 Clone & Configure

```
# Clone the repository
git clone https://github.com/your-org/healthcare-agent.git
cd healthcare-agent

# Copy environment template
cp config/.env.example config/.env
```

5.3 Environment Variables (config/.env)

```
# ── Azure OpenAI ──────────────────────────────────────
AZURE_OPENAI_API_KEY=your_azure_openai_key_here
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/
AZURE_OPENAI_API_VERSION=2024-02-01
AZURE_OPENAI_DEPLOYMENT_NAME=gpt-4o
AZURE_OPENAI_EMBEDDING_DEPLOYMENT=text-embedding-3-small

# ── Application ───────────────────────────────────────
APP_ENV=development
APP_SECRET_KEY=change-me-in-production-use-openssl-rand
APP_DEBUG=true
APP_LOG_LEVEL=INFO

# ── Database ──────────────────────────────────────────
DATABASE_URL=sqlite:///./data/health.db

# ── Storage Paths ─────────────────────────────────────
PDF_STORAGE_PATH=./data/pdfs
FAISS_INDEX_PATH=./data/faiss_index
DRUG_KNOWLEDGE_PATH=./data/drug_knowledge

# ── Embedding ─────────────────────────────────────────
EMBEDDING_CHUNK_SIZE=500
EMBEDDING_CHUNK_OVERLAP=50
FAISS_TOP_K=5

# ── CORS ──────────────────────────────────────────────
CORS_ORIGINS=["http://localhost:3000","http://127.0.0.1:3000"]
```

5.4 Config Loader (config/settings.py)

```
from pydantic_settings import BaseSettings
from functools import lru_cache
from typing import List

class Settings(BaseSettings):
    # Azure OpenAI
    azure_openai_api_key: str
    azure_openai_endpoint: str
```

```
    azure_openai_api_version: str = '2024-02-01'
    azure_openai_deployment_name: str = 'gpt-4o'
    azure_openai_embedding_deployment: str = 'text-embedding-3-small'

    # Application
    app_env: str = 'development'
    app_secret_key: str
    app_debug: bool = False
    app_log_level: str = 'INFO'

    # Database
    database_url: str = 'sqlite:///./data/health.db'

    # Storage
    pdf_storage_path: str = './data/pdfs'
    faiss_index_path: str = './data/faiss_index'
    drug_knowledge_path: str = './data/drug_knowledge'

    # Embedding
    embedding_chunk_size: int = 500
    embedding_chunk_overlap: int = 50
    faiss_top_k: int = 5

    # CORS
    cors_origins: List[str] = ['http://localhost:3000']

    class Config:
        env_file = 'config/.env'
        env_file_encoding = 'utf-8'

@lru_cache()
def get_settings() -> Settings:
    return Settings()
```

6. Docker Setup — Phase 1 (Local)

6.1 docker-compose.yml

```
version: '3.9'

services:

  # ── Backend: FastAPI ──────────────────────────────
  backend:
    build:
      context: .
      dockerfile: docker/Dockerfile.backend
    container_name: hfa-backend
    env_file: config/.env
    ports:
      - '8000:8000'
    volumes:
      - ./data:/app/data          # persist PDFs + FAISS indexes
      - ./backend:/app/backend    # hot-reload in dev
    depends_on:
```

```
      - redis
    healthcheck:
      test: ['CMD', 'curl', '-f', 'http://localhost:8000/health']
      interval: 30s
      timeout: 10s
      retries: 3
    networks:
      - hfa-network

  # ── Frontend: React + Nginx ──────────────────────────
  frontend:
    build:
      context: .
      dockerfile: docker/Dockerfile.frontend
    container_name: hfa-frontend
    ports:
      - '3000:80'
    depends_on:
      - backend
    networks:
      - hfa-network

  # ── Redis: Session / Rate Limiting (optional in dev) ─────
  redis:
    image: redis:7.2-alpine
    container_name: hfa-redis
    ports:
      - '6379:6379'
    networks:
      - hfa-network

networks:
  hfa-network:
    driver: bridge

volumes:
  hfa-data:
```

6.2 Dockerfile.backend

```
FROM python:3.11-slim-bookworm

# Install system dependencies for PyMuPDF
RUN apt-get update && apt-get install -y \
    build-essential libmupdf-dev curl \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Install Python dependencies (cached layer)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application source
COPY backend/ ./backend/
COPY config/ ./config/
COPY data/ ./data/
```

```
# Create writable data directories
RUN mkdir -p data/pdfs data/faiss_index data/drug_knowledge

EXPOSE 8000

# Use Uvicorn with auto-reload in development
CMD ["uvicorn", "backend.app:app", \
     "--host", "0.0.0.0", \
     "--port", "8000", \
     "--reload", \
     "--log-level", "info"]
```

6.3 Dockerfile.frontend

```
# ── Stage 1: Build React app ──────────────────────
FROM node:20-alpine AS builder
WORKDIR /app
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ .
RUN npm run build

# ── Stage 2: Serve with Nginx ─────────────────────
FROM nginx:1.25-alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY docker/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

6.4 Nginx Configuration (docker/nginx.conf)

```
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;

    # React SPA — fallback to index.html for client-side routing
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Proxy API calls to FastAPI backend
    location /api/ {
        proxy_pass http://backend:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 300s;   # long timeout for LLM responses
        proxy_buffering off;       # enable streaming SSE
    }
}
```

6.5 Running the Stack

```
# Build all images and start the stack
docker compose up --build

# Run in detached mode (background)
docker compose up -d --build

# View logs
docker compose logs -f backend
docker compose logs -f frontend

# Stop everything
docker compose down

# Stop and remove volumes (full reset)
docker compose down -v

# Rebuild a single service
docker compose up -d --build backend

# Access points
# Frontend:     http://localhost:3000
# Backend API:  http://localhost:8000
# API Docs:     http://localhost:8000/docs   (Swagger UI)
# ReDoc:        http://localhost:8000/redoc
# Health check: http://localhost:8000/health
```

> **First-Run Initialization**
> On first startup, the backend automatically runs init_db.py which creates all SQLite tables. Data persists across container restarts because the ./data directory is bind-mounted as a Docker volume.

7. FastAPI Application

7.1 Application Entry Point (backend/app.py)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager
import structlog

from backend.db.database import engine
from backend.db import models
from backend.routes import upload, chat, insights, fitness
from config.settings import get_settings

log = structlog.get_logger()
settings = get_settings()

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: create tables, warm up embeddings model
    log.info('Starting Healthcare Agent API')
```

```
    models.Base.metadata.create_all(bind=engine)
    yield
    # Shutdown
    log.info('Shutting down')

app = FastAPI(
    title='Healthcare + Fitness Agent API',
    description='AI-powered health analysis and fitness planning',
    version='1.0.0',
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

app.include_router(upload.router, prefix='/upload', tags=['Upload'])
app.include_router(chat.router,   prefix='/chat',   tags=['Chat'])
app.include_router(insights.router, prefix='/insights', tags=['Insights'])
app.include_router(fitness.router,  prefix='/fitness',  tags=['Fitness'])

@app.get('/health')
async def health_check():
    return {'status': 'ok', 'version': '1.0.0'}
```

7.2 Pydantic Schemas (backend/models/schemas.py)

```
from pydantic import BaseModel, Field
from typing import List, Optional, Literal

class UserProfile(BaseModel):
    user_id: str
    age: int = Field(ge=1, le=120)
    gender: Literal['male', 'female', 'other']
    height_cm: float = Field(ge=50, le=250)
    weight_kg: float = Field(ge=20, le=300)

class HealthParameter(BaseModel):
    test_name: str
    value: float
    unit: str
    reference_range: str
    status: Literal['Normal', 'Low', 'High', 'Critical']

class UploadResponse(BaseModel):
    message: str
    file_id: str
    parameters_extracted: int

class ChatRequest(BaseModel):
    user_id: str
    session_id: str
    query: str
```

```
class ChatResponse(BaseModel):
    answer: str
    disclaimer: str = 'This is not medical advice. Consult a qualified physician.'
    sources: List[str] = []

class InsightResponse(BaseModel):
    user_id: str
    risk_score: int
    risk_level: Literal['Low', 'Moderate', 'High', 'Critical']
    flagged_parameters: List[HealthParameter]
    conditions: List[str]
    recommendation: str

class FitnessPlanRequest(BaseModel):
    user_id: str
    include_diet: bool = True
    include_workout: bool = True

class FitnessPlanResponse(BaseModel):
    workout: List[str]
    diet: List[str]
    restrictions: List[str]
    lifestyle: List[str]
    disclaimer: str = 'Consult a physician before starting any new exercise
regimen.'
```

8. Database Layer (SQLite + SQLAlchemy)

8.1 Database Connection (backend/db/database.py)

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from config.settings import get_settings

settings = get_settings()

engine = create_engine(
    settings.database_url,
    connect_args={'check_same_thread': False},  # Required for SQLite
    echo=settings.app_debug,
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    """FastAPI dependency — yields a DB session per request."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

8.2 ORM Models (backend/db/models.py)

```
from sqlalchemy import Column, Integer, String, Float, Text, DateTime
from sqlalchemy.sql import func
from backend.db.database import Base

class User(Base):
    __tablename__ = 'users'
    user_id    = Column(String, primary_key=True, index=True)
    email      = Column(String, unique=True, nullable=True)
    age        = Column(Integer)
    gender     = Column(String)
    height_cm  = Column(Float)
    weight_kg  = Column(Float)
    created_at = Column(DateTime, server_default=func.now())
    updated_at = Column(DateTime, onupdate=func.now())

class HealthRecord(Base):
    __tablename__ = 'health_records'
    id              = Column(Integer, primary_key=True, autoincrement=True)
    user_id         = Column(String, index=True)
    parameter       = Column(String)
    value           = Column(Float)
    unit            = Column(String)
    reference_range = Column(String)
    status          = Column(String)    # Normal | Low | High | Critical
    file_id         = Column(String)
    recorded_at     = Column(DateTime, server_default=func.now())

class ChatSession(Base):
    __tablename__ = 'chat_sessions'
    session_id  = Column(String, primary_key=True)
    user_id     = Column(String, index=True)
    started_at  = Column(DateTime, server_default=func.now())

class ChatMessage(Base):
    __tablename__ = 'chat_messages'
    id         = Column(Integer, primary_key=True, autoincrement=True)
    session_id = Column(String, index=True)
    user_id    = Column(String)
    role       = Column(String)    # user | assistant
    content    = Column(Text)
    created_at = Column(DateTime, server_default=func.now())
```

9. PDF Ingestion Pipeline

9.1 Upload Route (backend/routes/upload.py)

```
from fastapi import APIRouter, UploadFile, File, Depends, HTTPException
from sqlalchemy.orm import Session
import uuid, os
from backend.db.database import get_db
from backend.services.pdf_service import process_pdf
from backend.models.schemas import UploadResponse
from config.settings import get_settings
```

```
router = APIRouter()
settings = get_settings()

@router.post('/{user_id}', response_model=UploadResponse)
async def upload_pdf(
    user_id: str,
    file: UploadFile = File(...),
    db: Session = Depends(get_db),
):
    if not file.filename.endswith('.pdf'):
        raise HTTPException(400, 'Only PDF files are accepted')
    if file.size and file.size > 20 * 1024 * 1024:  # 20 MB limit
        raise HTTPException(413, 'File too large (max 20 MB)')

    file_id = str(uuid.uuid4())
    file_path = os.path.join(
        settings.pdf_storage_path,
        f'{user_id}_{file_id}.pdf'
    )
    os.makedirs(settings.pdf_storage_path, exist_ok=True)

    content = await file.read()
    with open(file_path, 'wb') as f:
        f.write(content)

    result = await process_pdf(file_path, user_id, file_id, db)
    return UploadResponse(
        message='PDF processed successfully',
        file_id=file_id,
        parameters_extracted=result['count'],
    )
```

9.2 PDF Service (backend/services/pdf_service.py)

```
import fitz          # PyMuPDF
import pdfplumber
import json
from langchain_text_splitters import RecursiveCharacterTextSplitter
from backend.agents.doc_agent import extract_parameters
from backend.services.embedding_service import upsert_to_faiss
from backend.db import models
from config.settings import get_settings

settings = get_settings()

def extract_text_pymupdf(file_path: str) -> str:
    doc = fitz.open(file_path)
    text = ''
    for page in doc:
        text += page.get_text()
    doc.close()
    return text.strip()

def extract_tables_pdfplumber(file_path: str) -> str:
    table_text = ''
    with pdfplumber.open(file_path) as pdf:
        for page in pdf.pages:
            tables = page.extract_tables()
```

```
            for table in tables:
                for row in table:
                    if row:
                        table_text += ' | '.join([str(c or '') for c in row]) + '\n'
    return table_text

async def process_pdf(file_path, user_id, file_id, db):
    # 1. Extract text using both parsers
    raw_text = extract_text_pymupdf(file_path)
    table_data = extract_tables_pdfplumber(file_path)
    combined = raw_text + '\n\n[TABLES]\n' + table_data

    # 2. LLM extraction of structured parameters
    parameters = await extract_parameters(combined)

    # 3. Persist to SQLite
    for p in parameters:
        record = models.HealthRecord(
            user_id=user_id, file_id=file_id,
            parameter=p['test_name'], value=float(p['value']),
            unit=p['unit'], reference_range=p['reference_range'],
            status=p.get('status', 'Unknown')
        )
        db.add(record)
    db.commit()

    # 4. Chunk text and embed into FAISS
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=settings.embedding_chunk_size,
        chunk_overlap=settings.embedding_chunk_overlap
    )
    chunks = splitter.split_text(combined)
    await upsert_to_faiss(user_id, chunks)

    return {'count': len(parameters), 'chunks': len(chunks)}
```

10. FAISS Vector Store

10.1 Embedding Service (backend/services/embedding_service.py)

```
import os
from langchain_community.vectorstores import FAISS
from langchain_openai import AzureOpenAIEmbeddings
from config.settings import get_settings

settings = get_settings()

def get_embeddings() -> AzureOpenAIEmbeddings:
    return AzureOpenAIEmbeddings(
        azure_deployment=settings.azure_openai_embedding_deployment,
        azure_endpoint=settings.azure_openai_endpoint,
        api_key=settings.azure_openai_api_key,
        api_version=settings.azure_openai_api_version,
    )
```

```
def get_index_path(user_id: str) -> str:
    path = os.path.join(settings.faiss_index_path, user_id)
    os.makedirs(path, exist_ok=True)
    return path

async def upsert_to_faiss(user_id: str, chunks: list[str]) -> None:
    embeddings = get_embeddings()
    index_path = get_index_path(user_id)

    if os.path.exists(os.path.join(index_path, 'index.faiss')):
        # Load existing index and merge
        vs = FAISS.load_local(index_path, embeddings,
                              allow_dangerous_deserialization=True)
        vs.add_texts(chunks)
    else:
        # Create new index
        vs = FAISS.from_texts(chunks, embeddings)

    vs.save_local(index_path)

def load_retriever(user_id: str):
    embeddings = get_embeddings()
    index_path = get_index_path(user_id)
    if not os.path.exists(os.path.join(index_path, 'index.faiss')):
        return None
    vs = FAISS.load_local(index_path, embeddings,
                          allow_dangerous_deserialization=True)
    return vs.as_retriever(
        search_type='similarity',
        search_kwargs={'k': settings.faiss_top_k}
    )
```

11. LangGraph Agent Design

11.1 Shared LLM Factory (backend/utils/llm_factory.py)

```
from langchain_openai import AzureChatOpenAI
from config.settings import get_settings
from functools import lru_cache

@lru_cache(maxsize=1)
def get_llm() -> AzureChatOpenAI:
    settings = get_settings()
    return AzureChatOpenAI(
        azure_deployment=settings.azure_openai_deployment_name,
        azure_endpoint=settings.azure_openai_endpoint,
        api_key=settings.azure_openai_api_key,
        api_version=settings.azure_openai_api_version,
        temperature=0.3,
        max_tokens=2000,
    )
```

11.2 Prompt Templates (backend/utils/prompt_templates.py)

```
SYSTEM_GUARDRAIL = """
You are a healthcare AI assistant. IMPORTANT RULES:
1. Never provide a definitive medical diagnosis.
2. Always recommend consulting a qualified physician for high-risk results.
3. Base your analysis only on the provided health data.
4. Flag any life-threatening values with CRITICAL: prefix.
5. Use clear, plain language — avoid medical jargon where possible.
"""

DOC_EXTRACTION_PROMPT = """
Extract all lab test results from the following medical document text.
Return a JSON array where each item has EXACTLY these fields:
- test_name: string
- value: number
- unit: string
- reference_range: string (e.g. '70-100 mg/dL')
- status: one of ['Normal', 'Low', 'High', 'Critical']

Document text:
{text}

Return ONLY the JSON array, no other text.
"""

RISK_ANALYSIS_PROMPT = """
{system}

Patient Profile:
- Age: {age} | Gender: {gender} | BMI: {bmi:.1f}

Flagged Health Parameters (outside reference ranges):
{flagged_params}

Detected Conditions: {conditions}

Provide a concise risk summary (3-4 sentences) and key recommendations.
"""

FITNESS_PLAN_PROMPT = """
{system}

Generate a personalized health plan for this patient:
- Age: {age} | BMI: {bmi:.1f} | Risk Level: {risk_level}
- Medical Restrictions: {restrictions}

Return ONLY valid JSON with this exact structure:
{{
  "workout": ["exercise 1", "exercise 2", ...],
  "diet": ["diet recommendation 1", ...],
  "restrictions": ["restriction 1", ...],
  "lifestyle": ["tip 1", ...]
}}
"""
```

11.3 Agent 1 — Intake Agent (backend/agents/intake_agent.py)

```
from dataclasses import dataclass
import math

@dataclass
class IntakeResult:
    bmi: float
    bmi_category: str
    bmr: float          # Basal Metabolic Rate (kcal/day)

def run_intake_agent(age: int, gender: str,
                     height_cm: float, weight_kg: float) -> IntakeResult:
    """Pure Python — no LLM call needed for baseline vitals."""
    bmi = weight_kg / ((height_cm / 100) ** 2)

    if bmi < 18.5:   category = 'Underweight'
    elif bmi < 25.0: category = 'Normal'
    elif bmi < 30.0: category = 'Overweight'
    else:            category = 'Obese'

    # Mifflin-St Jeor equation
    if gender == 'male':
        bmr = 10 * weight_kg + 6.25 * height_cm - 5 * age + 5
    else:
        bmr = 10 * weight_kg + 6.25 * height_cm - 5 * age - 161

    return IntakeResult(bmi=round(bmi, 2), bmi_category=category, bmr=round(bmr))
```

11.4 Agent 2 — Document Analysis Agent (backend/agents/doc_agent.py)

```
import json
from langchain_core.messages import HumanMessage, SystemMessage
from backend.utils.llm_factory import get_llm
from backend.utils.prompt_templates import DOC_EXTRACTION_PROMPT

async def extract_parameters(text: str) -> list[dict]:
    """Use GPT-4o to extract structured lab parameters from raw PDF text."""
    llm = get_llm()
    prompt = DOC_EXTRACTION_PROMPT.format(text=text[:6000])  # token guard

    response = await llm.ainvoke([
        SystemMessage(content='You are a medical data extraction assistant.'),
        HumanMessage(content=prompt),
    ])

    try:
        raw = response.content.strip()
        # Strip markdown fences if present
        if raw.startswith('```'):
            raw = raw.split('```')[1]
            if raw.startswith('json'): raw = raw[4:]
        return json.loads(raw.strip())
    except json.JSONDecodeError:
        return []
```

11.5 Agent 3 — Health Risk Agent (backend/agents/risk_agent.py)

```
from backend.db.models import HealthRecord
from backend.utils.llm_factory import get_llm
from backend.utils.prompt_templates import RISK_ANALYSIS_PROMPT, SYSTEM_GUARDRAIL
from langchain_core.messages import HumanMessage

RISK_WEIGHTS = {
    'glucose':       {'High': 30, 'Low': 20, 'Critical': 60},
    'hba1c':         {'High': 35, 'Low': 15, 'Critical': 70},
    'cholesterol':   {'High': 20, 'Low': 10, 'Critical': 40},
    'triglycerides': {'High': 20, 'Low': 10, 'Critical': 40},
    'creatinine':    {'High': 25, 'Low': 15, 'Critical': 50},
    'hemoglobin':    {'High': 15, 'Low': 25, 'Critical': 45},
}

def compute_risk_score(records: list[HealthRecord]) -> tuple[int, str]:
    score = 0
    for r in records:
        key = r.parameter.lower()
        weights = RISK_WEIGHTS.get(key, {})
        score += weights.get(r.status, 0)

    if score == 0:   level = 'Low'
    elif score < 40: level = 'Moderate'
    elif score < 70: level = 'High'
    else:            level = 'Critical'
    return min(score, 100), level

async def run_risk_agent(user_id, age, gender, bmi, records, conditions):
    flagged = [r for r in records if r.status != 'Normal']
    flagged_text = '\n'.join(
        f'  - {r.parameter}: {r.value} {r.unit} [{r.reference_range}] → {r.status}'
        for r in flagged
    ) or 'None — all parameters within range'

    prompt = RISK_ANALYSIS_PROMPT.format(
        system=SYSTEM_GUARDRAIL,
        age=age, gender=gender, bmi=bmi,
        flagged_params=flagged_text,
        conditions=', '.join(conditions) or 'None detected',
    )
    llm = get_llm()
    response = await llm.ainvoke([HumanMessage(content=prompt)])
    return response.content
```

11.6 Agent 4 — Disease Pattern Agent (backend/agents/disease_agent.py)

```
from backend.db.models import HealthRecord

def get_param(records: list[HealthRecord], name: str) -> float | None:
    name_lower = name.lower()
    for r in records:
        if name_lower in r.parameter.lower():
```

```
            return r.value
    return None

def run_disease_agent(records: list[HealthRecord]) -> list[str]:
    """Rule-based pattern detection — zero LLM cost."""
    conditions = []
    glucose = get_param(records, 'glucose')
    trig    = get_param(records, 'triglyceride')
    hdl     = get_param(records, 'hdl')
    ldl     = get_param(records, 'ldl')
    hba1c   = get_param(records, 'hba1c')
    creat   = get_param(records, 'creatinine')
    hgb     = get_param(records, 'hemoglobin')
    bp_sys  = get_param(records, 'systolic')

    # Metabolic Syndrome (≥3 of 5 criteria)
    met_criteria = sum([
        bool(glucose and glucose > 100),
        bool(trig and trig > 150),
        bool(hdl and hdl < 40),
        bool(bp_sys and bp_sys > 130),
    ])
    if met_criteria >= 3:
        conditions.append('Metabolic Syndrome Risk')

    # Pre-diabetes
    if glucose and 100 <= glucose <= 125:
        conditions.append('Pre-diabetes (IFG)')
    if hba1c and 5.7 <= hba1c <= 6.4:
        conditions.append('Pre-diabetes (HbA1c)')

    # Diabetes screening
    if glucose and glucose > 126:
        conditions.append('Possible Diabetes — Refer to physician')

    # Dyslipidemia
    if ldl and ldl > 160:
        conditions.append('High LDL — Dyslipidemia Risk')

    # Anemia screening
    if hgb and hgb < 12:
        conditions.append('Possible Anemia')

    # Kidney function
    if creat and creat > 1.4:
        conditions.append('Elevated Creatinine — Kidney function review needed')

    return conditions
```

11.7 Agent 5 — Fitness Plan Agent (backend/agents/fitness_agent.py)

```
import json
from langchain_core.messages import HumanMessage
from backend.utils.llm_factory import get_llm
from backend.utils.prompt_templates import FITNESS_PLAN_PROMPT, SYSTEM_GUARDRAIL

def get_fitness_level(bmi: float, risk_level: str) -> str:
```

```
    if risk_level == 'Critical': return 'Sedentary Recovery'
    if bmi < 25 and risk_level == 'Low': return 'Intermediate'
    if bmi >= 30 or risk_level == 'High': return 'Beginner'
    return 'Beginner-Intermediate'

async def run_fitness_agent(
    age: int, bmi: float, risk_level: str, conditions: list[str]
) -> dict:
    fitness_level = get_fitness_level(bmi, risk_level)
    restrictions = ', '.join(conditions) if conditions else 'None'

    prompt = FITNESS_PLAN_PROMPT.format(
        system=SYSTEM_GUARDRAIL,
        age=age, bmi=bmi,
        risk_level=f'{risk_level} (Fitness Level: {fitness_level})',
        restrictions=restrictions,
    )
    llm = get_llm()
    response = await llm.ainvoke([HumanMessage(content=prompt)])

    raw = response.content.strip()
    if raw.startswith('```'):
        raw = raw.split('```')[1]
        if raw.startswith('json'): raw = raw[4:]

    try:
        plan = json.loads(raw.strip())
    except json.JSONDecodeError:
        plan = {
            'workout':      ['Walking 30 min/day', 'Light stretching'],
            'diet':         ['Balanced meals', 'Reduce processed sugar'],
            'restrictions': ['Consult physician before starting'],
            'lifestyle':    ['7-8 hours sleep', 'Stress management'],
        }
    return plan
```

11.8 LangGraph State Graph (backend/agents/graph.py)

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List, Optional

class AgentState(TypedDict):
    user_id:      str
    age:          int
    gender:       str
    bmi:          float
    records:      list
    conditions:   List[str]
    risk_score:   int
    risk_level:   str
    risk_summary: str
    fitness_plan: Optional[dict]

def build_health_graph() -> StateGraph:
    from backend.agents.intake_agent  import run_intake_agent
    from backend.agents.disease_agent import run_disease_agent
    from backend.agents.risk_agent    import compute_risk_score, run_risk_agent
    from backend.agents.fitness_agent import run_fitness_agent
```

```
    async def node_disease(state: AgentState) -> AgentState:
        state['conditions'] = run_disease_agent(state['records'])
        return state

    async def node_risk(state: AgentState) -> AgentState:
        score, level = compute_risk_score(state['records'])
        state['risk_score'] = score
        state['risk_level'] = level
        summary = await run_risk_agent(
            state['user_id'], state['age'], state['gender'],
            state['bmi'], state['records'], state['conditions']
        )
        state['risk_summary'] = summary
        return state

    async def node_fitness(state: AgentState) -> AgentState:
        plan = await run_fitness_agent(
            state['age'], state['bmi'],
            state['risk_level'], state['conditions']
        )
        state['fitness_plan'] = plan
        return state

    graph = StateGraph(AgentState)
    graph.add_node('disease_detection', node_disease)
    graph.add_node('risk_analysis',     node_risk)
    graph.add_node('fitness_planning',  node_fitness)

    graph.set_entry_point('disease_detection')
    graph.add_edge('disease_detection', 'risk_analysis')
    graph.add_edge('risk_analysis',     'fitness_planning')
    graph.add_edge('fitness_planning',  END)

    return graph.compile()
```

12. RAG Chat System

12.1 Chat Route (backend/routes/chat.py)

```
from fastapi import APIRouter, Depends
from fastapi.responses import StreamingResponse
from sqlalchemy.orm import Session
from backend.db.database import get_db
from backend.db import models
from backend.services.embedding_service import load_retriever
from backend.utils.llm_factory import get_llm
from backend.utils.guardrails import wrap_with_disclaimer
from backend.models.schemas import ChatRequest
from langchain.chains import RetrievalQA
from langchain_core.prompts import PromptTemplate

router = APIRouter()

RAG_PROMPT = PromptTemplate(
```

```
    input_variables=['context', 'question'],
    template="""
You are a healthcare AI assistant. Use ONLY the context below to answer.
If the context does not contain the answer, say 'I cannot find that in your health
records.'
Never provide a medical diagnosis. Always recommend consulting a physician for
concerning values.

Context from health records:
{context}

Patient Question: {question}

Answer:"""
)

@router.post('/')
async def chat(request: ChatRequest, db: Session = Depends(get_db)):
    retriever = load_retriever(request.user_id)
    if retriever is None:
        return {'answer': 'No health records found. Please upload a PDF first.',
                'disclaimer': wrap_with_disclaimer('')}

    llm = get_llm()
    qa_chain = RetrievalQA.from_chain_type(
        llm=llm,
        chain_type='stuff',
        retriever=retriever,
        chain_type_kwargs={'prompt': RAG_PROMPT}
    )

    response = await qa_chain.ainvoke({'query': request.query})
    answer = response['result']

    # Persist message to DB
    for role, content in [('user', request.query), ('assistant', answer)]:
        db.add(models.ChatMessage(
            session_id=request.session_id,
            user_id=request.user_id,
            role=role, content=content
        ))
    db.commit()

    return {
        'answer': answer,
        'disclaimer': 'This is not medical advice. Consult a qualified physician.',
    }
```

12.2 Guardrails (backend/utils/guardrails.py)

```
MEDICAL_DISCLAIMER = (
    '\n\n---\n'
    'DISCLAIMER: This information is generated by an AI assistant for educational
purposes '
    'only and does not constitute medical advice, diagnosis, or treatment. '
    'Always consult a qualified healthcare professional before making any health
decisions.'
)
```

```
CRITICAL_KEYWORDS = ['diabetes', 'cardiac', 'kidney failure', 'critical',
                     'emergency', 'hospital', 'immediate']

def wrap_with_disclaimer(response: str) -> str:
    return response + MEDICAL_DISCLAIMER

def check_critical(response: str) -> bool:
    """Flag responses that may need urgent referral."""
    lower = response.lower()
    return any(kw in lower for kw in CRITICAL_KEYWORDS)

def apply_guardrails(response: str) -> dict:
    is_critical = check_critical(response)
    if is_critical:
        response = ('URGENT: Please consult a physician immediately.\n\n' +
response)
    return {
        'content': wrap_with_disclaimer(response),
        'requires_doctor': is_critical,
    }
```

13. API Endpoints Reference

Layer	Technology	Purpose / Notes
Method	Endpoint	Description
POST	/upload/{user_id}	Upload a PDF health report for a user
POST	/chat/	Ask a question using RAG over user's health records
GET	/insights/{user_id}	Get risk score, conditions, and LLM risk summary
POST	/fitness/plan	Generate personalized workout and diet plan
POST	/users/	Create or update a user profile
GET	/users/{user_id}	Retrieve a user profile
GET	/health	Health check endpoint (used by Docker healthcheck)
GET	/docs	Swagger UI — interactive API documentation

14. Python Dependencies (requirements.txt)

```
# ── Web Framework ──────────────────────────────────
fastapi==0.111.0
uvicorn[standard]==0.29.0
```

```
python-multipart==0.0.9

# ── Data Validation ──────────────────────────────
pydantic==2.7.1
pydantic-settings==2.2.1

# ── Database ─────────────────────────────────────
sqlalchemy==2.0.30

# ── AI / LLM ─────────────────────────────────────
langchain==0.2.0
langchain-openai==0.1.7
langchain-community==0.2.0
langchain-text-splitters==0.2.0
langgraph==0.1.5

# ── Vector Store ─────────────────────────────────
faiss-cpu==1.8.0

# ── PDF Processing ───────────────────────────────
pymupdf==1.24.3
pdfplumber==0.11.0

# ── Utilities ────────────────────────────────────
python-dotenv==1.0.1
structlog==24.1.0
httpx==0.27.0
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
```

15. Testing Strategy

15.1 Unit Tests — Risk Scoring

```
# tests/unit/test_risk_scoring.py
import pytest
from backend.agents.risk_agent import compute_risk_score
from backend.db.models import HealthRecord

def make_record(param, status):
    r = HealthRecord()
    r.parameter = param
    r.value = 0.0
    r.unit = 'unit'
    r.reference_range = '0-0'
    r.status = status
    return r

def test_low_risk():
    records = [make_record('glucose', 'Normal'), make_record('hdl', 'Normal')]
    score, level = compute_risk_score(records)
    assert level == 'Low'
    assert score == 0

def test_high_risk_glucose():
```

```
    records = [make_record('glucose', 'High'), make_record('hba1c', 'High')]
    score, level = compute_risk_score(records)
    assert score >= 40
    assert level in ['High', 'Critical']

def test_critical_cap():
    records = [make_record('glucose', 'Critical'), make_record('hba1c', 'Critical')]
    score, level = compute_risk_score(records)
    assert score <= 100   # score is capped at 100
    assert level == 'Critical'
```

15.2 Integration Test — Full PDF Pipeline

```
# tests/integration/test_upload_pipeline.py
import pytest
from httpx import AsyncClient
from backend.app import app

@pytest.mark.asyncio
async def test_upload_and_insights():
    async with AsyncClient(app=app, base_url='http://test') as client:
        # 1. Upload a sample PDF
        with open('tests/fixtures/sample_lab.pdf', 'rb') as f:
            response = await client.post(
                '/upload/test-user-001',
                files={'file': ('lab.pdf', f, 'application/pdf')}
            )
        assert response.status_code == 200
        data = response.json()
        assert data['parameters_extracted'] > 0

        # 2. Query insights
        resp = await client.get('/insights/test-user-001')
        assert resp.status_code == 200
        insight = resp.json()
        assert 'risk_level' in insight
        assert insight['risk_level'] in ['Low', 'Moderate', 'High', 'Critical']
```

15.3 Running Tests

```
# Install dev dependencies
pip install -r requirements.dev.txt

# Run all tests
pytest tests/ -v

# Run only unit tests
pytest tests/unit/ -v

# Run with coverage report
pytest tests/ --cov=backend --cov-report=html

# Run inside Docker
docker compose exec backend pytest tests/ -v
```

16. Security & Guardrails

16.1 Dev Mode Security Checklist

- User ID isolation: all data namespaced under user_id — FAISS index at data/faiss_index/{user_id}/, SQLite filtered by user_id column
- API key management: Azure OpenAI key stored in .env, never committed to git. Add config/.env to .gitignore
- File validation: only .pdf MIME type accepted, 20 MB max, content-type verified server-side
- Input sanitization: all user inputs pass through Pydantic v2 validators before processing
- Medical guardrails: every LLM response has the disclaimer appended and critical keyword detection
- Rate limiting: Redis-backed slowapi rate limiter scaffolded (10 req/min per user_id in dev)
- Prompt injection defense: user input is never directly interpolated into system prompts — always passed as HumanMessage

Production Security Note (Phase 2)
Before deploying to Azure: implement JWT authentication (python-jose), enable HTTPS via Azure Application Gateway, rotate the APP_SECRET_KEY to a cryptographically random value (openssl rand -hex 32), and enable Azure Key Vault for secret management.

17. Development Milestones

Phase	Timeline	Deliverables
Phase 1	Days 1–3	Docker Compose stack, SQLite schema, FastAPI skeleton, health check, .env configuration, Nginx reverse proxy
Phase 2	Days 4–6	PDF upload endpoint, PyMuPDF + pdfplumber extraction, Doc Analysis Agent, SQLite persistence, FAISS indexing
Phase 3	Days 7–9	Disease Pattern Agent (rule engine), Risk Scoring Engine, LangGraph StateGraph wiring, Insights endpoint
Phase 4	Days 10–12	RAG Chat system, Fitness Plan Agent, Fitness endpoint, medical guardrails on all responses
Phase 5	Days 13–15	React UI (upload, chat, insights, plan pages), Zustand store, API client, Tailwind styling
Phase 6	Days 16–18	Unit + integration tests, pytest coverage, Drug Interaction

		Engine (FAISS-based), Drug knowledge base loading
Phase 7	Future	Azure Container Instances deployment, Azure Key Vault, Application Gateway (TLS), WhatsApp integration

18. Azure Deployment — Phase 2 Notes

18.1 Container Registry Push

```
# Tag images for Azure Container Registry
az acr login --name yourregistry

docker tag hfa-backend yourregistry.azurecr.io/hfa-backend:v1.0
docker tag hfa-frontend yourregistry.azurecr.io/hfa-frontend:v1.0

docker push yourregistry.azurecr.io/hfa-backend:v1.0
docker push yourregistry.azurecr.io/hfa-frontend:v1.0
```

18.2 Production Changes Required

- Replace SQLite with Azure SQL (PostgreSQL) or Cosmos DB for multi-instance support
- Replace local FAISS with Azure Cognitive Search or Qdrant on ACI for persistent vector storage
- Replace local PDF store with Azure Blob Storage (mount as volume or use azure-storage-blob SDK)
- Add JWT authentication middleware with Azure AD B2C integration
- Configure Azure Application Gateway for TLS termination and load balancing
- Move all secrets to Azure Key Vault — remove .env file entirely
- Enable Application Insights for distributed tracing and LLM token usage monitoring

> **FAISS in Production**
> FAISS is not designed for multi-process shared access. In Phase 2, migrate to Qdrant (self-hosted on ACI) or Azure AI Search for concurrent multi-user vector search. The embedding_service.py interface is already abstracted to make this swap seamless.

19. Troubleshooting Guide

Container fails to start

Check: docker compose logs backend. Common causes: missing .env file, invalid Azure OpenAI key, or port 8000/3000 already in use. Run docker compose down then docker compose up --build.

PDF extraction returns zero parameters

The PDF may be image-based (scanned). PyMuPDF extracts embedded text only. For scanned PDFs, add pytesseract for OCR pre-processing. Verify the PDF is text-searchable by opening it and attempting Ctrl+A to select all text.

FAISS index not found error

The user has not uploaded any PDF yet. The /chat endpoint will return a 'No health records found' message. Ensure the data/faiss_index directory exists (created by Docker volumes on first run).

Azure OpenAI 429 Rate Limit errors

Add exponential backoff retry logic using tenacity: @retry(wait=wait_exponential(min=1, max=10), stop=stop_after_attempt(3)). Check your Azure OpenAI quota in the Azure Portal.

SQLite database locked error

SQLite has limited concurrency. This occurs when multiple async tasks write simultaneously. Ensure all DB writes use the same SessionLocal via the get_db() dependency injection. In production, migrate to PostgreSQL.

> **Next Steps**
> Generate the complete FastAPI application code using this guide as the spec. Build the LangGraph agent workflow visualization. Create the React UI component library. Set up CI/CD with GitHub Actions for Docker image builds.